Computer Architecture Industry Lecture-2 Scribe on Caches and Prefetching

Vanamala Aman Preetham, CS23BTECH11063
Panshul Jindal, AI23BTECH11018
Tanishq Arvind Chirame, CS23BTECH11013
Mohammed Abdur Rehman, AI23BTECH11015

18 November 2024

Acknowledgements: We thank Dr. Dipika Deb from Intel for visiting us and conducting this lecture.

Brief Overview of Cache:

We started with a brief overview of cache.

Important of Cache:

SRAM hardware is in between front end and DRAM. By front end, we mean the Fetch and Decode Units. CPU is in GHz and DRAM is in MHz. So, there is a speed mismatch. Caches try to cover this speed gap by hiding the long DRAM Latency. The advantage of using cache is that it increases performance, which is measured in terms of IPC (Instructions per Cycle).

Locality:

Caches take advantage of temporal and spatial locality. Take the below word addressable cache as an example.

Levels of Cache:

The levels of cache are L1, L2 and L3. L3 is most of the times Last Level Cache (LLC). It helps if we have bigger memory.

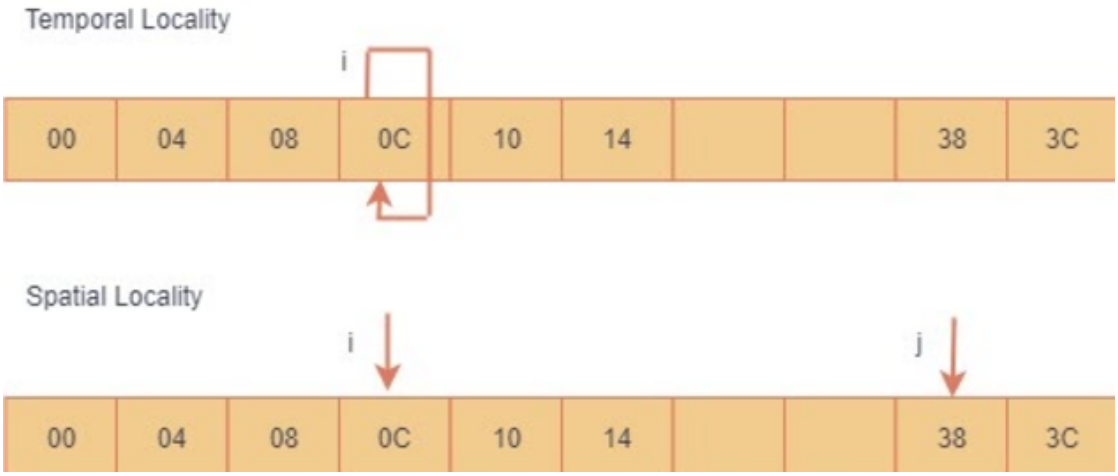


Figure 1: Localities

Disadvantages of increasing cache size:

Power Issue:

More cache size increases the power consumed, which is measured in Performance/Watt.

Area:

More cache size increases the area required to store it.

Disadvantages of increasing cache levels:

While increasing levels might seem advantageous due to the fact that it hides the memory latency further, it has the following disadvantage that on cache miss, multiple cache levels are indexed and AMAT increases when we increase the number of cache levels.

$$\text{AMAT} = \text{Hit time} + \text{Miss Rate} * \text{Miss Latency}$$

Pipelining Latency:

Pipelining latency increases on increasing cache size. We have 5 stages in the MIPS pipeline. They are Fetch, Decode, Execute, Memory and Write back. Here L1 Instruction cache is present near the Fetch and Decode units and L1 Data cache is present near the Memory and Write back Units. As the cache size of L1 Instruction cache increases, Insruction fetch and decode time increases. Similarly if L1 Data cache size increases, memory access and write back time increases.

Prefetching:

It is a technique which predicts what addresses will be requested in future by the code and brings those lines to the cache. The prefetcher observes the memory access pattern and then generates a prefetch request.

Cache prefetching can either fetch data or instructions into cache. Data access patterns show less regularity than instruction patterns, accurate data prefetching is generally more challenging than instruction prefetching.

Software-based prefetching is typically accomplished by having the compiler analyze the code and insert additional "prefetch" instructions in the program during compilation itself. Hardware-based prefetching is typically accomplished by having a dedicated hardware mechanism.

The first mainstream microprocessors to use some form of instruction prefetch were the Intel 8086 (six bytes) and the Motorola 68000 (four bytes).

An Example:

Suppose we have A, B, C, D, E as blocks in the memory and we have a cache which can contain two blocks only. If we access B, then it is a miss and instead of fetching only B, we also fetch C in the cache so that next time when we access C, it is a cache hit.

Types of Prefetching:

There are two broad categories of prefetching namely hardware and software prefetching. Software prefetching is handled by the compiler. We mainly discussed about hardware prefetching in the lecture:

Hardware prefetching:

It uses latency hiding techniques. Cache contains demand lines and prefetched lines. Demand lines are the lines that are requested by the code and prefetched lines are all the lines that are prefetched for future accesses.

Advantage of using a prefetcher:

Using a prefetcher reduces number of cache misses. This decreases the number of DRAM accesses which consequently reduces AMAT.

Disadvantage of using a prefetcher - Cache Pollution:

It might be that prefetching evicts some important or frequently used cache lines which will result in more number of misses than intended and the code needs to again bring the important lines from DRAM.

Different parameters of consideration to determine prefetcher's efficiency:

Prefetch Accuracy:

$$\text{Prefetch Accuracy} = \frac{\text{Number of Prefetched Cache Hits}}{\text{Number of Prefetched Cache Blocks}}$$

It is computed over few cycles. If prefetcher accuracy is good it is kept, otherwise it is asked not to prefetch.

Timeliness:

Let a prefetch request go to DRAM but is still in prefetching stage. But before that, if we try to access the memory address which is being prefetched, it will be a cache miss. Good qualitative timeliness measures help to address this problem. So prefetcher needs to fetch the line early enough to avoid such misses. For knowing the right time to trigger prefetching, timeliness is important.

Consider a for loop where each iteration takes 3 cycles to execute and the 'prefetch' operation takes 12 cycles. This implies that for the prefetched data to be useful, the system must start the prefetch $12/3 = 4$ iterations prior to its usage to maintain timeliness.

Prefetch Coverage:

$$\text{Prefetch Coverage} = \frac{\text{Number of Cache Misses Covered by Prefetching}}{\text{Total Cache Misses}}$$

It measures how many cache misses are reduced due to prefetching. Even if accuracy is good, coverage need not be good. For example if accuracy is 100%, but coverage can be 70%. It is because prefetched lines might be evicted before it even got accessed.

Aggressiveness:

The number of prefetches initiated for a memory access is called the aggressiveness of the prefetcher. It is also called prefetch degree or stride.

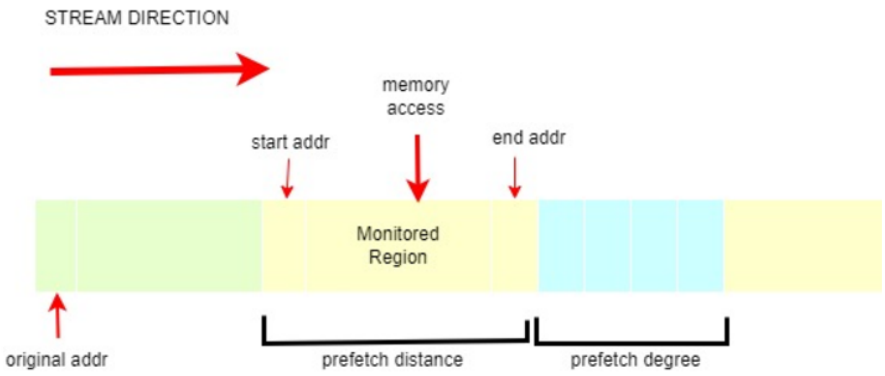


Figure 2: Prefetching Aggressiveness

Irregular spatial strides:

In this case, the delta between the addresses of consecutive memory accesses is variable but still follows a pattern. Some prefetchers designs exploit this property to predict and prefetch for future accesses.

Next Line Prefetcher:

It does not look into access pattern. It always prefetches the next line. It is used when streaming behavior is observed. The stride is +1 for this prefetcher. In implementation, the bit [6] of demand address is flipped and the cache line is prefetched. For example, Demand address: 0xffff00, Prefetch address: 0xffff40

VLDP Prefetcher

Reference: [2]

Introduction

- **Pattern 1**: $A, A-24, A+1, A-23, A+2, A-22, \ldots$

$$\text{Delta} = (-24, +25, -24, +25, \ldots)$$

- **Pattern 2**: $A, A+1, A+10, A+2, A+3, A+11, A+3, A+4, A+13$

$$0, 1, 9, 2, 1, 9, 3, 1, 9 \quad (\text{Base: } +1, +9)$$

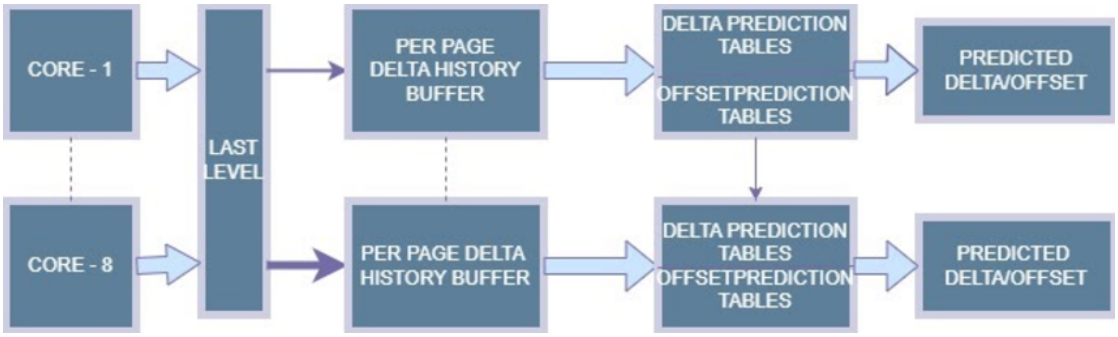


Figure 3: VLDP Mechanism

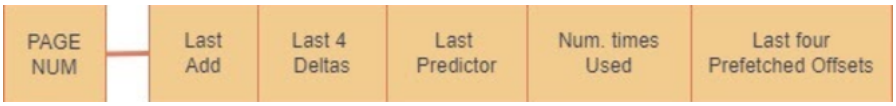


Figure 4: Delta History Buffer Entry:

It is used in detecting complex patterns. It predicts future memory requests on different length of input histories by using multiple prediction tables where each successive table corresponds to a longer history length.

This maximizes prefetch coverage and accuracy, by preferring to use long histories to make highly accurate prefetches, and using shorter histories to fill in the gaps when long histories are unavailable.

PAE: Prefetch Activation Event PAE is triggered on

- A cache miss
- A cache hit on DHB (Delta History Buffer)

Prediction/Prefetching Mechanism

1. Per Page Delta History Buffer (DHB)

Purpose:
The Delta History Buffer (DHB) tracks delta histories for recently accessed pages.
Structure:
DHB maintains separate histories for each active physical page.
Each entry corresponds to a tracked page and contains:

1. Page number
2. Page offset of the last address accessed in this page

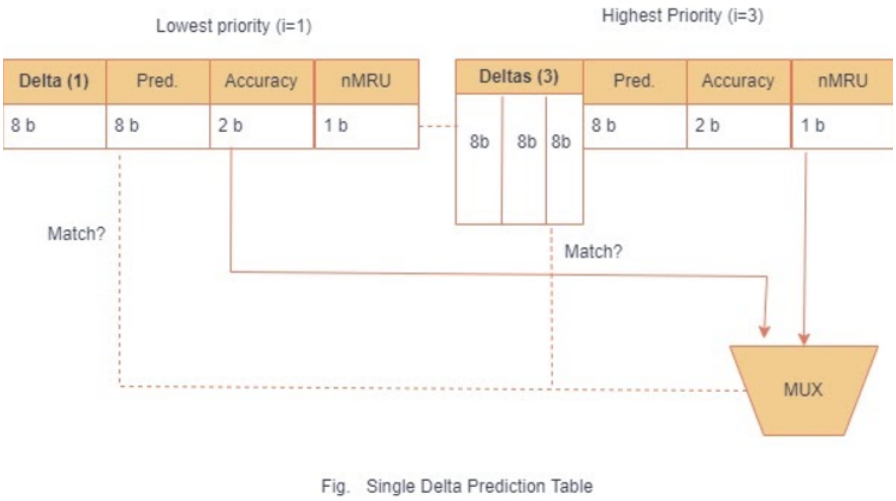


Fig. Single Delta Prediction Table

Figure 5: Single Delta Prediction Table

3. Sequence of up to 4 recently observed deltas
4. the DPT level used for the latest delta prediction
5. the number of times this page has been used
6. sequence of up to 4 recently prefetched offsets

Only PAEs can cause the state stored in the DHB to change.

Operation:

When a page is accessed, the DHB is updated:
On a DHB miss (a new page), an existing entry is evicted using a not-Most-Recently-Used (nMRU) policy.
On a DHB hit, the delta between the current access and the last recorded offset is computed and added to the delta history. This history is then used to look up predictions in the Delta Prediction Table (DPT).

When a PAE occurs, we look for delta history matches in all tables. When searching for a delta to prefetch, VLDP prefers to use predictions that come from DPT tables that track longer histories.

OPT handles pages accessed only once, where delta history is unavailable. Tracks initial page offsets and predicts the next likely offset for the first access.

DS-Patch : Dual Spatial Pattern Prefetcher:

Reference: [1]

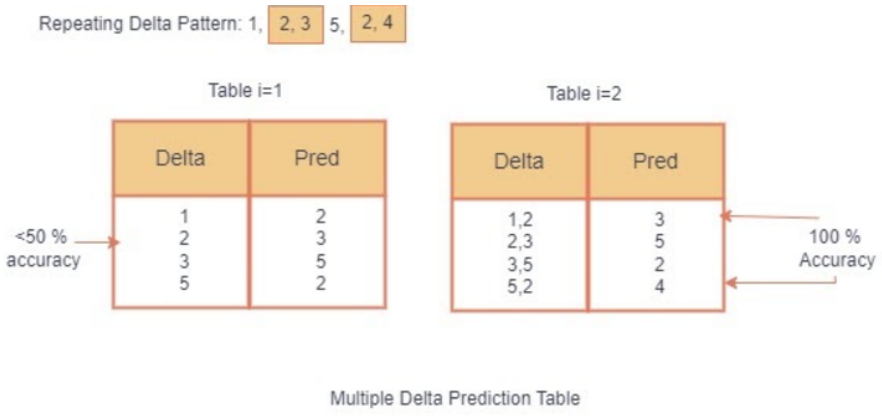


Figure 6: Multiple Delta Prediction Table

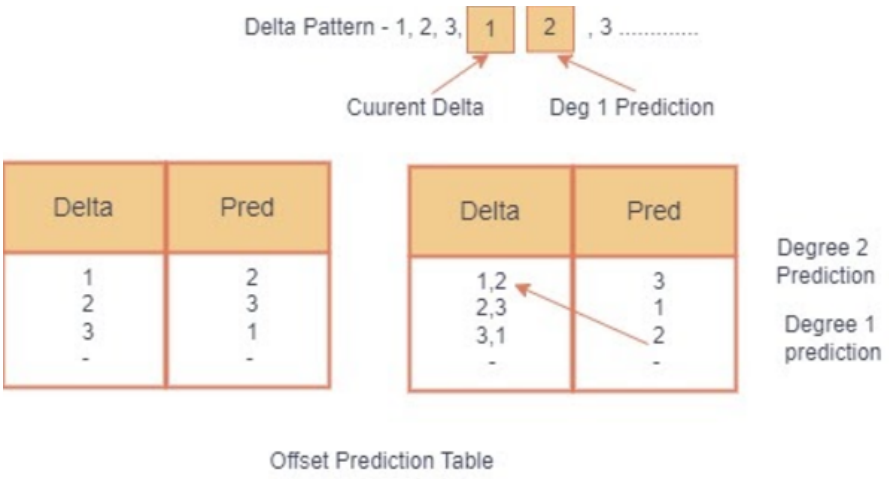


Figure 7: Offset Prediction Table

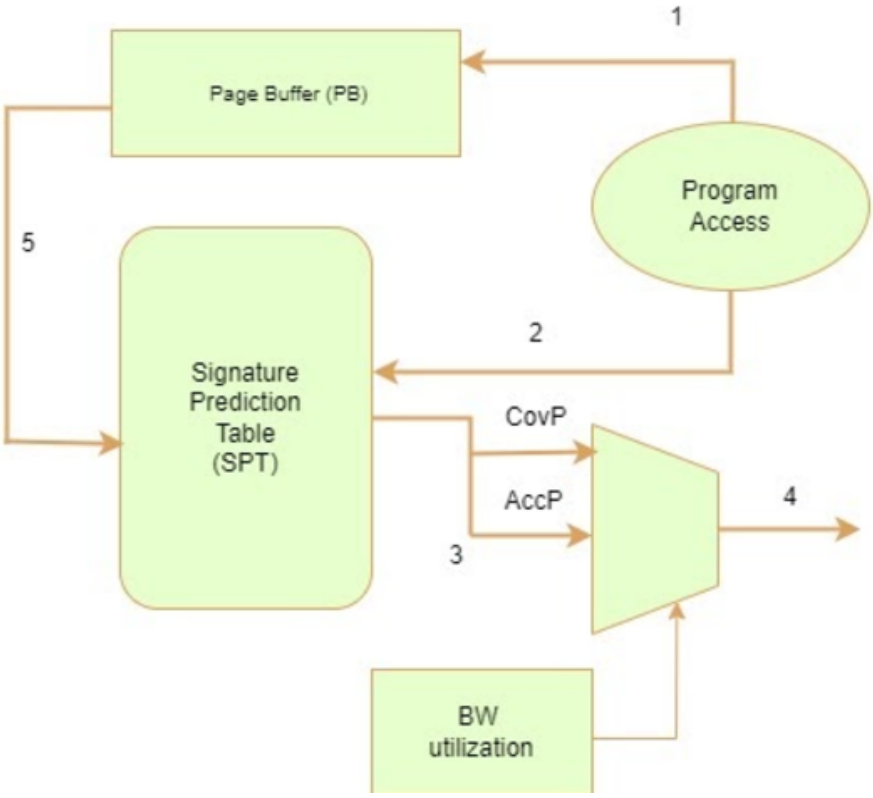


Figure 8: DS-Patch

The Dual Spatial Pattern Prefetcher (DSPP) is an advanced prefetching mechanism designed to enhance memory performance in computer systems by predicting future memory accesses.

Dual Approach:

DSPP combines spatial prefetching with advanced pattern recognition techniques. It identifies contiguous data blocks (spatial locality) as well as recurring access patterns in memory references to improve accuracy.

Dynamic Pattern Prediction:

DSPP adjusts to changes in memory access patterns dynamically, ensuring consistent performance even as workloads evolve.

Implementation Complexity:

While DSPP improves prediction accuracy, it also introduces some hardware overhead compared to simpler prefetchers like next-line prefetchers.
Hardware requirements include metadata storage for tracking patterns and logic units for pattern recognition.

Works for many tasks:

It's great for games, simulations, and data-heavy apps.

Some other topics discussed:

Requirement of simple prefetchers despite having complex prefetchers:

VLDP can detect +1 stride. But it needs history to create the table. So we need simpler implementations which can detect +1 stride with less amount of history needed to predict.

Discussion regarding using ML model in prefetching:

Perceptron-based prefetcher uses one neuron to determine prefetching. It came up during the discussion of using ML model in prefetching. But ML models require training which adds a lot of overhead which needs to be kept in mind for designing such methods.

Cache warm-up:

It is the phase when cache starts getting filled with blocks. It occurs when system is first powered on or after it has been idle for sometime. Initially, there will be many cache misses because the data has not yet been stored in the cache. Once the cache is filled, the system can reach its optimal performance with most number of accesses being cache hits.

Codenames for different INTEL microarchitectures:

Lakes:

It is used for PCs, client products etc.

Rapids:

It is used for servers.

Coves:

It is used for core components.

Future and Conclusion:

Disintegreted Dies:

Usually a single die contains all the components like CPUs, caches, RAM etc., but this causes a problem that when one component gets damaged then the whole die needs to be replaced. This problem is tackled by having multiple dies with each containing one component such as die with caches, die with I/O etc. This makes it easier if any component gets damaged, then only that part needs to be replaced without affecting any other die. But a new problem now arises that we have to connect them together. They need a common language on which work is still going on.

New communication backbone:

New ways are being invented through which we can fetch from slow DRAM memory faster. The ways include photonic, wireless and quantum communication.

References

[1] Rahul Bera, Anant V. Nori, Onur Mutlu, and Sreenivas Subramoney. Dspatch: Dual spatial pattern prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '52, page 531–544, New York, NY, USA, 2019. Association for Computing Machinery. URL: https://doi.org/10.1145/3352460.3358325.

[2] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramonian, Chris Wilkerson, Seth H. Pugsley, and Zeshan Chishti. Efficiently prefetching complex address patterns. In *Proceedings of the 48th International Symposium on Microarchitecture*, MICRO-48, page 141–152, New York, NY, USA, 2015. Association for Computing Machinery. URL: https://doi.org/10.1145/2830772.2830793.